SIMATS ENGINEERING
ASSESSMENT TOOL

COURSE NAME: Theory of Computation **COURSE CODE: CSA1304**

COURSE OUTCOME COVERED

CO2: De**ign Finite Automata DFA, NFA, ε-NFA and demonstrate their equivalence for recognizing regular languages. (BL:3)**

Assessment Tool Weightage: 40%

QUESTIONS: 5 **Total Marks:25**
Duration : 1 Hour

AT2 - Design Task

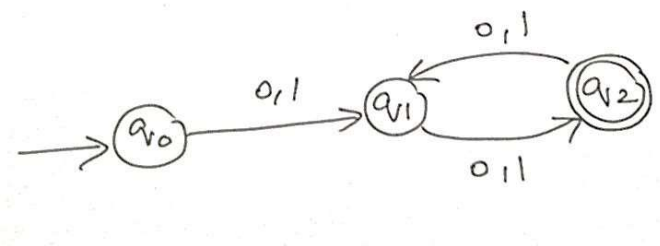
Code of Conduct:

I GIFTSON KEVIN MATTHEW.Y (192511167)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: GIFTSON KEVIN MATTHEW.Y

SIMATS ENGINEERING
ASSESSMENT TOOL

1. Design the regular expression for the language represented by the given automata



2. Consider the following grammar

S->aSc|T|U|e

T-> bTc|e

U->aUb|e

A} What is the language defined by the grammar?

B) Give the derivation tree for the word aaabbc

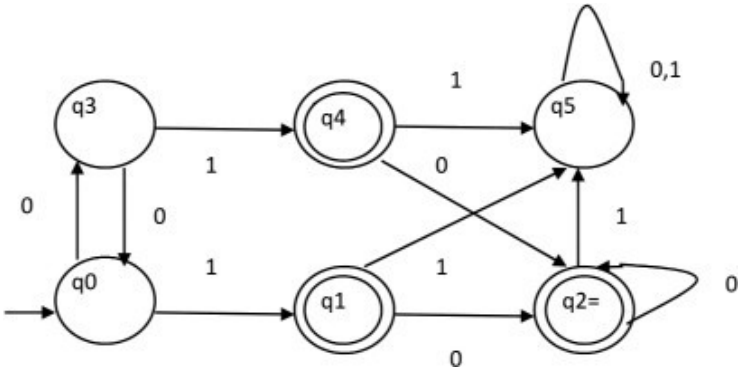
C) A Grammar is Ambiguous if there exists two distinct derivations for a word. Is the above grammar ambiguous? Justify.

3. Design a non deterministic finite automata with ε-transitions to represent the language denoted by the regular expression
00*1+11*0

4. **I)**Show that if L1 and L2 are regular then (L1+l2) also regular

Ii)Is the language { a^p| p is prime} regular? Justify

5. Design minimized automata, Consider the DFA given below. Find the states that are equivalent and construct a minimum state automata equivalent to the given automata. Draw the minimized automata, write its transition table and mark initial and final states



SIMATS ENGINEERING
ASSESSMENT TOOL

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	**30**	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	**25**	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	**20**	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	**15**	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	**10**	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

SIMATS ENGINEERING
ASSESSMENT TOOL

Question 1. Design the regular expression for the language represented by the given automata.

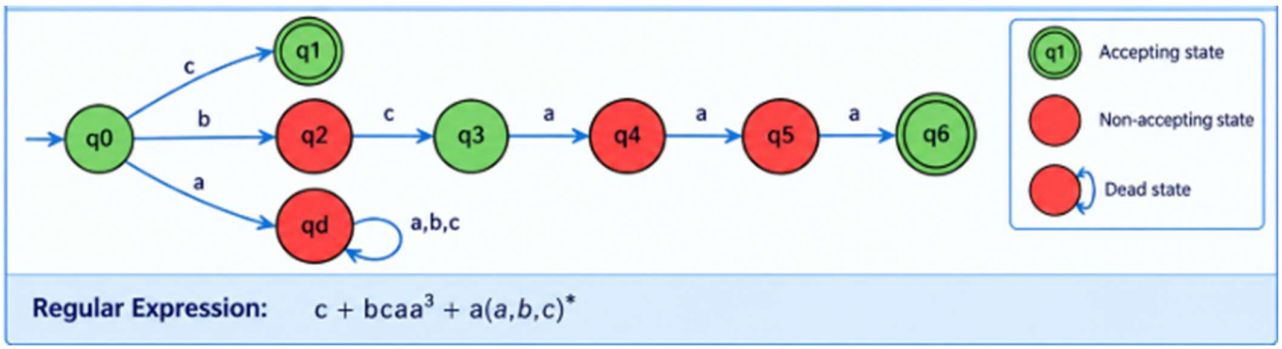
Concept:

A regular expression represents a language using symbols and operators. The main operators are:

- Union (+): Represents a choice between two languages.
- Concatenation: Represents symbols occurring one after another.
- Kleene star (*): Represents zero or more repetitions of a symbol or expression.

Procedure to find the regular expression:

1. Identify the initial state and the final state of the given automaton.
2. Observe the input symbols on each transition.
3. Identify the different paths from the initial state to the final state.
4. Write a regular expression for each accepting path.
5. Combine the expressions using union (+) and simplify the result.



Conclusion: The required regular expression is obtained by representing all possible input strings that take the automaton from the initial state to a final state.

Question 2. Consider the following grammar

Given grammar:

[S rightarrow aSc mid T mid U mid varepsilon]

[T rightarrow bTc mid varepsilon]

[U rightarrow aUb mid varepsilon]

A) What is the language defined by the grammar?

Answer:

The language generated by a grammar is the set of all strings that can be derived from its start symbol (S). Consider each production separately.

1. Production of S

[S rightarrow aSc]

This production adds one a to the beginning and one c to the end. Repeating it (n) times produces strings of the form

[a^nSc^n]

2. Production of T

[T rightarrow bTc mid varepsilon]

Each application adds one b at the beginning and one c at the end. Therefore,

[L(T)={b^n c^n mid ngeq 0}]

Examples are (varepsilon, bc, bbcc, bbbccc).

3. Production of U

[U rightarrow aUb mid varepsilon]

Each application adds one a at the beginning and one b at the end. Therefore,

[L(U)={a^n b^n mid n geq 0}]

Examples are(varepsilon, ab,aabb,aaabbb).

Since (S) can select (T), (U), or (varepsilon), and can also repeatedly apply (S rightarrow aSc), the complete language is

[boxed{ L(S)={a^i w c^mid igeq0,win L(T)cup L(U)cup{varepsilon}} }]

Here, (w) is either a string of the form (b^n c^n), a string of the form (a^n b^n), or the empty string.

Conclusion: The grammar generates strings by surrounding a string derived from (T), (U), or (varepsilon) with equal numbers of a symbols on the left and c symbols on the right.

B) Give the derivation tree for the word aaabbc.

Answer:

We need to check whether aaabbc can be derived from the given grammar.

The word starts with three a symbols. Using \(S\rightarrow aSc\) three times gives

[SRightarrow aSc]

[Rightarrow aaScc]

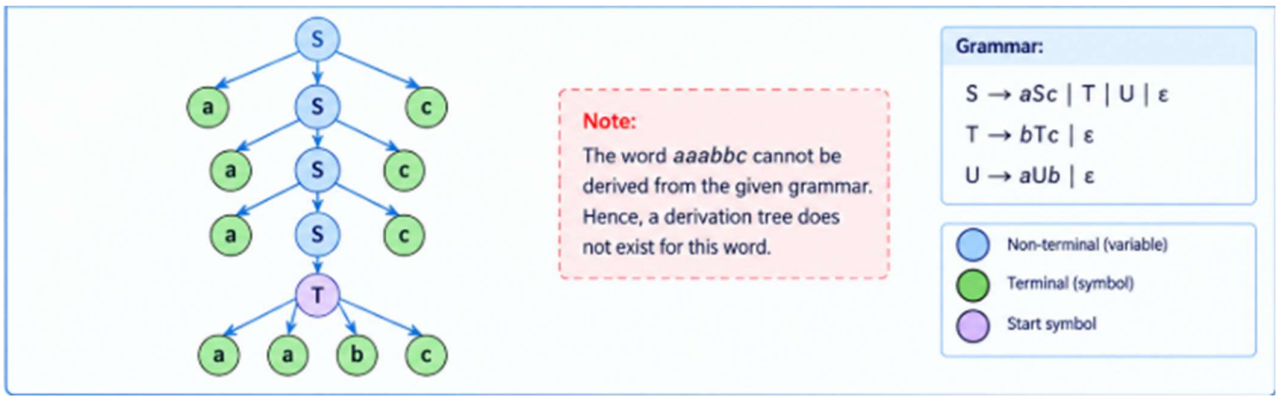
[Rightarrow aaaSccc]

At this stage, the derived string must have at least three c symbols at the end. The remaining \(S\) can produce \(T\), \(U\), or \(\varepsilon\), but none of these choices removes the three trailing c symbols.

Therefore, aaabbc, which contains only one c, cannot be derived from the given grammar.

[boxed{aaabbcnotin L(S)}]

Conclusion: A valid derivation tree for aaabbc does not exist for this grammar. The question may contain a typo in the given word or grammar.



C) Is the above grammar ambiguous? Justify.

Answer:

A context-free grammar is called ambiguous if at least one string in its language has two distinct parse trees, or equivalently, two distinct leftmost derivations.

Consider the empty string \(\varepsilon\).

It can be derived directly using

[SRightarrowvarepsilon]

It can also be derived through \(T\):

[S Rightarrow T\Rightarrow\varepsilon]

And through (U):

[S Rightarrow U Rightarrow\varepsilon]

These are distinct derivations of the same string.

Therefore, under the standard definition of ambiguity based on distinct parse trees, we should check whether these derivations correspond to distinct parse trees. They do: the first uses the direct production from \(S\), the second uses \(S\rightarrow T\), and the third uses \(S\rightarrow U\).

Hence,

[\boxed{\text{The grammar is ambiguous.}} \]

Justification: The same string, \(\varepsilon\), has different parse trees because it can be derived through three different production paths.

SIMATS ENGINEERING
ASSESSMENT TOOL

Question 3. Design an NFA with ε-transitions for the regular expression \(00^*1+11^*0\).

Answer:

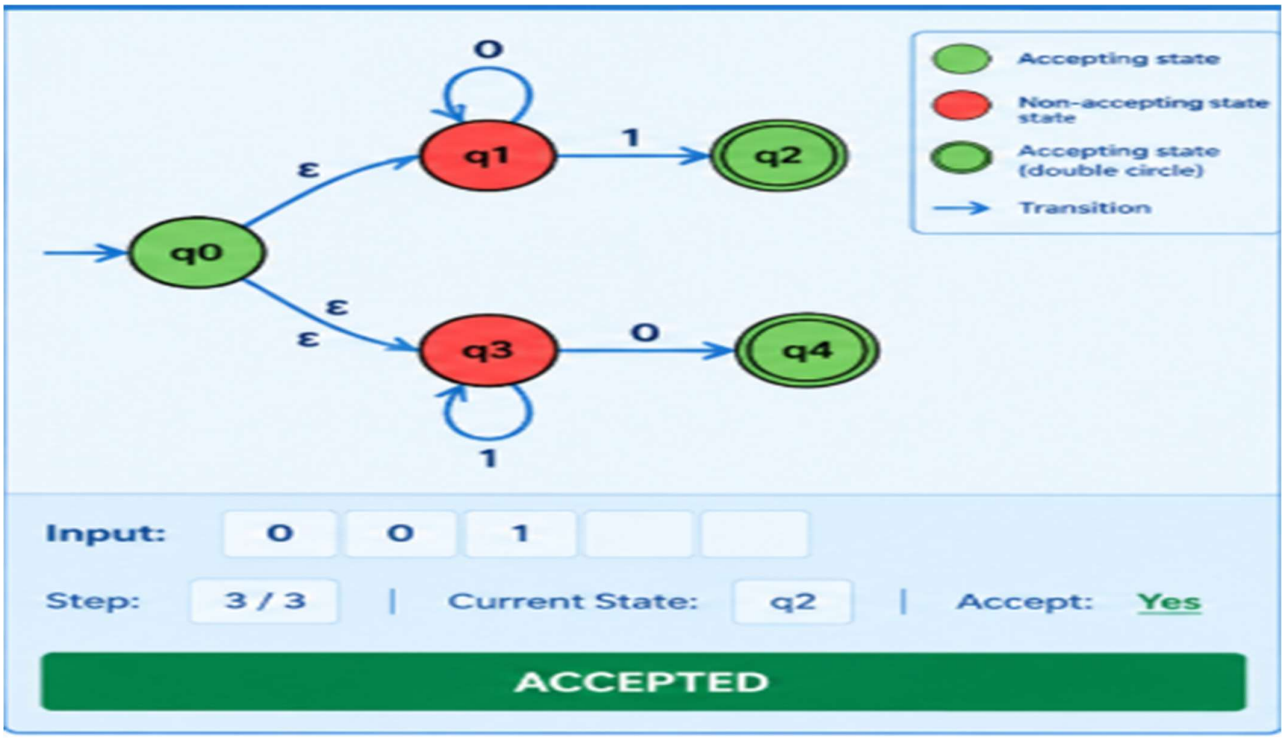
The given regular expression is

\[R=00^*1+11^*0 \]

The + operator represents union, so the automaton must accept either of these two patterns:

- \(00^*1\): one or more 0 symbols followed by one 1.
- \(11^*0\): one or more 1 symbols followed by one 0.

We can construct an ε-NFA by splitting the initial state into two branches using ε-transitions.



Step 1: Define the states

- (q_0): Initial state.
- (q_1): State for the first branch, reading zeros.
- (q_2): Final state for the first branch.
- (q_3): State for the second branch, reading ones.
- (q_4): Final state for the second branch.

Step 2: Define the transitions

Current state	Input	Next state
(q_0)	(varepsilon)	(q_1,q_3)
(q_1)	0	(q_1)
(q_1)	1	(q_2)

(q_3)	1	(q_3)
(q_3)	0	(q_4)

Here, (q_2) and (q_4) are final states.

Step 3: Explain the working

For input 001:

[q_0\xrightarrow{\varepsilon}q_1 \xrightarrow{0}q_1 \xrightarrow{0}q_1 \xrightarrow{1}q_2]

The input is accepted.

For input 110:

[q_0\xrightarrow{\varepsilon}q_3 \xrightarrow{1}q_3 \xrightarrow{1}q_3 \xrightarrow{0}q_4]

The input is accepted.

Conclusion: The ε-NFA accepts strings consisting of one or more zeros followed by one 1, or one or more ones followed by one 0.

Question 4

I) Show that if (L_1) and (L_2) are regular, then (L_1+L_2) is also regular.

Answer:

Let (L_1) and (L_2) be two regular languages.

Since both languages are regular, there exist finite automata (M_1) and (M_2) that recognize them.

Let

[M_1=(Q_1,\Sigma,\delta_1,q_1,F_1)]

[M_2=(Q_2,\Sigma,\delta_2,q_2,F_2)]

To recognize the union (L_1\cup L_2), construct a new ε-NFA with a new initial state (q_s). Add ε-transitions from (q_s) to the initial states of (M_1) and (M_2).

The new automaton can choose either (M_1\) or (M_2\) without consuming any input symbol.

The final states are the final states of both automata:

[F=F_1\cup F_2 \]

The constructed automaton accepts a string if it is accepted by either (M_1\) or \(M_2\).

Therefore,

[L(M)=L_1\cup L_2 \]

Since an ε-NFA is equivalent in expressive power to a DFA, the union is also recognized by a finite automaton.

Hence,

[\boxed{L_1,L_2\text{ regular}\ightarrow L_1\cup L_2\text{ regular}}

Conclusion: Regular languages are closed under the union operation.

SIMATS ENGINEERING
ASSESSMENT TOOL

II) Is the language (\{a^p\mid p\text{ is prime}}) regular? Justify.

Answer:

The language is

L=\{a^p\mid p\text{ is prime}}]

It contains strings consisting only of a, where the length of the string is a prime number.

Examples include

[aa,\ aaa,\ aaaaa,\ aaaaaaa]

These strings have lengths (2,3,5,7\), which are prime numbers.

Assume, for contradiction, that\(L\) is regular. By the pumping lemma, there exists a pumping length \(n\).

Choose a prime number (p\geq n\) and consider the string

[w=a^p]

According to the pumping lemm, (w=xyz), where

- (|xy|\leq p\),
- (|y|>0\),
- (xy^iz\in L\) for every (i\geq0\).

Since the string contains only a, let (|y|=k\), where \(k\geq1\).

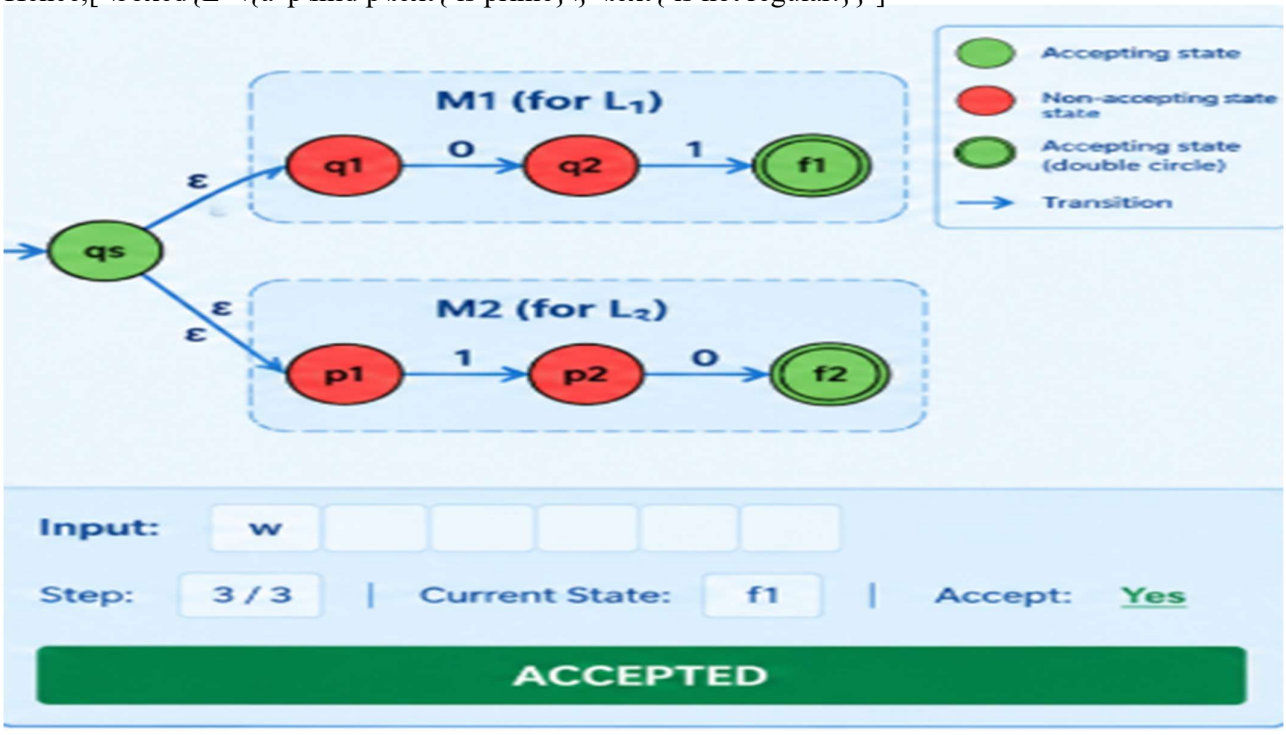
Choose (i=p+1\). The resulting string has length

[|xy^{p+1}z|=p+pk=p(1+k)]

Since (p) is prime and (k\geq1\), both \(p\) and \(1+k\) are integers greater than 1. Therefore, (p(1+k)) is composite, not prime.

This contradicts the pumping lemma requirement that the pumped string must belong to \(L\).

Hence,[\boxed{L=\{a^p\mid p\text{ is prime}\} \text{ is not regular.}}]



Conclusion: The language of strings containing a prime number of a symbols is not regular because a finite automaton cannot recognize exactly the prime lengths.

Question 5. Design minimized automata.

The question asks you to find equivalent states in the given DFA, construct the minimum-state equivalent DFA, draw it, write its transition table, and mark the initial and final states.

However, the DFA diagram is not visible in the extracted text of your PDF, so the exact states and transition table cannot be determined yet.

Method to minimize a DFA:

1. Divide the states into two groups: final states and non-final states.
2. Check whether states in each group have transitions to different groups for the same input symbol.
3. Split any group whose states are distinguishable.
4. Repeat until no further splitting is possible.
5. Merge the remaining equivalent states.
6. Construct the minimized DFA and its transition table.

Two states are equivalent if every possible input string causes both states either to accept or to reject.

Transition table format:

State	Input 0	Input 1
State group 1	Next state	Next state
State group 2	Next state	Next state

The actual state names, transitions, and final states must be filled in from the original diagram.

